

Smart Transit Predictor

Northeast India

AI-Powered Road Safety Prediction System
Machine Learning + Real-Time Weather Data

Covering All 8 States | 13 Cities | 18 Routes

Technologies: FastAPI | XGBoost | Leaflet.js | Open-Meteo API

Language: Python 3.10+ | HTML/CSS/JavaScript

Table of Contents

1. Problem Statement
2. Features & User Experience
3. Coverage - All 8 NE States
4. How It Works (Architecture)
5. Technology Stack
6. Project Structure
7. ML Model Performance
8. API Documentation
9. Setup & Installation
10. Why This Project Stands Out
11. Future Improvements

1. Problem Statement

Northeast India - Mizoram, Manipur, Assam, Meghalaya, Nagaland, Arunachal Pradesh, Tripura, and Sikkim - has some of the most difficult and dangerous roads in India. Every year:

- Landslides block highways for days during the monsoon season

The Scenario:

Imagine you are traveling from Aizawl to Silchar and halfway through the journey, a landslide blocks the road. No one warned you! You are stranded on a remote mountain road with no cell service and no help.

Our Solution:

This app solves this problem. You enter your route and travel date, and the app tells you the risk level of that road based on real-time weather data and terrain analysis using Machine Learning.

2. Features & User Experience

What the User Sees

A stunning dark-themed website where you:

- Select a starting city (e.g., Aizawl)

Results Include:

- Risk Score - Safe (green), Caution (yellow), or Danger (red)

Design Features

- Premium dark theme with glassmorphism effects

3. Coverage - All 8 Northeast States

13 Cities

State	City	Elevation
Arunachal Pradesh	Itanagar	440m
Assam	Guwahati	55m
Assam	Silchar	35m
Assam	Tezpur	48m
Assam	Jorhat	116m
Assam	Dibrugarh	108m
Manipur	Imphal	786m
Meghalaya	Shillong	1496m
Mizoram	Aizawl	1132m
Nagaland	Kohima	1444m
Nagaland	Dimapur	260m
Sikkim	Gangtok	1650m
Tripura	Agartala	16m

18 Routes (53 Segments)

Route	Distance	Key Risk
Aizawl - Silchar	180 km	Heavy rainfall, steep
Aizawl - Guwahati	480 km	Landslide prone
Imphal - Guwahati	450 km	NH-2 landslide zones
Shillong - Guwahati	100 km	Fog, steep curves
Agartala - Silchar	170 km	Flooding prone
Imphal - Silchar	200 km	Mountain roads
Dimapur - Kohima	74 km	Extremely steep
Kohima - Imphal	135 km	Most dangerous NH-2
Guwahati - Tezpur	180 km	Brahmaputra floods
Guwahati - Jorhat	310 km	Kaziranga zone
Jorhat - Dibrugarh	130 km	River flooding
Tezpur - Itanagar	280 km	Mountain, landslide
Dibrugarh - Itanagar	480 km	Remote mountains
Guwahati - Gangtok	560 km	Mountain passes
Silchar - Shillong	330 km	Jaintia Hills
Agartala - Guwahati	590 km	Very long route
Jorhat - Kohima	190 km	Mountain ascent
Dimapur - Jorhat	150 km	Naga Hills

4. How It Works (Architecture)

Step 1: Weather Data (Open-Meteo API)

We fetch real-time weather data for each route segment's midpoint coordinates. This includes rainfall (last 24h + forecast), temperature, wind speed, and humidity. The Open-Meteo API is completely free and requires no API key.

Step 2: Terrain Data

We store terrain information for each of the 53 route segments, including slope angle, maximum elevation, road type (highway vs mountain road), and historical blockage frequency.

Step 3: Machine Learning (XGBoost)

An XGBoost classifier trained on 6,000+ synthetic samples predicts risk using 9 input features. The model outputs one of three classes: Safe (0), Caution (1), or Danger (2), along with confidence probabilities.

Input Features:

Feature	Description	Range
rainfall_24h	Rainfall in last 24 hours	0-120 mm
rainfall_forecast_24h	Expected rainfall next 24h	0-100 mm
temperature	Temperature	5-35 C
wind_speed	Wind speed	0-60 km/h
humidity	Humidity percentage	40-100%
slope	Terrain slope angle	5-45 degrees
road_type	Highway(0) / Mountain(1)	0 or 1
month	Month of travel	1-12
historical_blockages	Blockages per year	0-20

Step 4: Map Visualization (Leaflet.js)

An interactive map renders the selected route with color-coded polylines: green for Safe, yellow for Caution, and red/dashed for Danger. City markers with permanent tooltips are displayed. Users can click on segments to see risk details.

Step 5: Web Application (FastAPI)

The FastAPI backend receives the route and date, fetches weather data for all segments in parallel using `async/await`, runs the ML prediction for each segment, and returns a comprehensive JSON response with risk scores, reasons, weather data, and route information.

System Architecture:

```
Frontend (HTML/JS/Leaflet.js)
  |
  v
FastAPI Backend (app/main.py)
  |           |
  v           v
Open-Meteo    XGBoost Model
Weather API   (models/xgboost_model.pkl)
  |           |
  v           v
Weather +    Risk Prediction
Terrain --> + Reasons
```

5. Technology Stack

Technology	Purpose	Why Chosen
FastAPI	Backend framework	Async, fast, auto docs
XGBoost	ML prediction	Best for tabular data
Open-Meteo	Weather data	Free, no API key
Leaflet.js	Interactive maps	Lightweight, powerful
scikit-learn	ML utilities	Metrics, train/test
httpx	HTTP client	Async support
pandas	Data processing	CSV handling
HTML/CSS/JS	Frontend	Premium dark theme

6. Project Structure

```

smart-transit-predictor/
|-- app/
|   |-- __init__.py      (Package init)
|   |-- main.py          (FastAPI - 5 endpoints)
|   |-- model.py         (XGBoost model wrapper)
|   |-- routes.py        (13 cities, 18 routes)
|   |-- weather.py       (Open-Meteo async client)
|-- data/
|   |-- terrain.csv       (Terrain data)
|   |-- incidents.csv     (50 historical incidents)
|   |-- training_data.csv (6000 training samples)
|-- frontend/
|   |-- index.html        (Premium dark-themed SPA)
|   |-- style.css         (Glassmorphism, 1100+ lines)
|   |-- script.js         (Map + API, 490+ lines)
|-- models/
|   |-- xgboost_model.pkl (Trained XGBoost)
|-- train_model.py        (Training script)
|-- test_api.py           (API test script)
|-- requirements.txt      (Dependencies)
|-- README.md             (Documentation)

```

7. ML Model Performance

Overall Accuracy: 91.4%

Classification Report

Class	Precision	Recall	F1-Score	Support
Safe	0.95	0.93	0.94	395
Caution	0.89	0.89	0.89	455
Danger	0.91	0.93	0.92	350

Feature Importance

Feature	Importance	Percentage
rainfall_24h	0.3454	34.5%
road_type	0.1538	15.4%
slope	0.1244	12.4%
historical_blockages	0.1090	10.9%
rainfall_forecast_24h	0.0886	8.9%
humidity	0.0626	6.3%
month	0.0512	5.1%
wind_speed	0.0391	3.9%
temperature	0.0259	2.6%

Training Details

- Training samples: 6,000 (synthetically generated)

8. API Documentation

Method	Endpoint	Description
GET	/	Serves the frontend
GET	/api/cities	All 13 cities with coordinates
GET	/api/routes	All 18 routes with segments
GET	/api/destinations/{city}	Reachable cities
POST	/api/predict	Predict risk for route + date

Predict Request (POST /api/predict)

```
{
  "from_city": "Aizawl",
  "to_city": "Silchar",
  "date": "2026-07-15"
}
```

Predict Response

```
{
  "risk_level": 2,
  "risk_label": "Danger",
  "confidence": 0.987,
  "reasons": [
    "Heavy rainfall (45.2mm in 24h)",
    "Very steep terrain (32 deg slope)",
    "Peak monsoon season (July)"
  ],
  "segments": [...],
  "weather": {
    "rainfall_24h": 45.2,
    "temperature": 22.5,
    "wind_speed": 35.0,
    "humidity": 92.0
  }
}
```

9. Setup & Installation

Prerequisites

- Python 3.10 or higher

Installation Steps

Step 1: Navigate to the project

```
cd smart-transit-predictor
```

Step 2: Install dependencies

```
pip install -r requirements.txt
```

Step 3: Train the ML model (one-time, ~5 seconds)

```
python train_model.py
```

Step 4: Start the web server

```
python -m uvicorn app.main:app --reload --port 8000
```

Step 5: Open in browser

```
http://localhost:8000
```

10. Why This Project Stands Out

Typical Student Project	This Project
Movie recommender	Solves a REAL problem
Uses toy datasets	Uses real weather API data
Simple model only	Full ML pipeline
No visualization	Interactive map
Basic UI	Premium glassmorphism dark theme
Single file	12+ files, proper structure
No API	Full REST API with FastAPI
Not deployment ready	Production-ready code

11. Future Improvements

- Add more cities and routes across NE India